

# *Recurrent Neural Networks*

## *Mathematical Foundations*

*Vitor N. Cabral de Oliveira*

*[vnco@cin.ufpe.br](mailto:vnco@cin.ufpe.br)*

*August 1, 2026*

This chapter develops the mathematical framework for Recurrent Neural Networks (RNNs), which are designed to process sequential data. We formalise the RNN architecture, derive the forward propagation equations, and present backpropagation through time (BPTT). The problems of vanishing and exploding gradients are analysed, and the Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) are introduced as effective solutions.

*Contents*

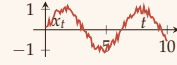
## Foundations of Sequence Modeling

Before diving into recurrent architectures, we must establish a rigorous framework for sequence data. Sequences are ordered collections of observations that arise naturally in time series, natural language, audio, video, and many other domains. Understanding their structure and the associated learning problems is essential.

### Definition and Notation

**Definition 0.1** (Sequence). A **sequence** of length  $T$  is an ordered tuple  $\mathbf{x}_{1:T} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T)$ , where each element  $\mathbf{x}_t$  belongs to a set  $\mathcal{X}$  (the **observation space**). When  $\mathcal{X} \subseteq \mathbb{R}^d$ , we speak of a **real-valued vector sequence**.

- If  $T$  is finite, the sequence is **finite**; if  $T = \infty$ , it is **infinite**.
- When the indices  $t$  are discrete (e.g.,  $t \in \mathbb{N}$ ), we have a **discrete-time sequence**. Continuous-time sequences are indexed by  $t \in \mathbb{R}$  but are less common in machine learning.



### Special cases

- **Univariate sequence**: each  $\mathbf{x}_t \in \mathbb{R}$  (scalar). Example: daily temperature.
- **Multivariate sequence**: each  $\mathbf{x}_t \in \mathbb{R}^d$ ,  $d > 1$ . Example: joint angles of a robotic arm over time.
- **Symbolic sequence**:  $\mathcal{X}$  is a finite set of symbols (e.g., characters, words). Example: natural language text.

### Probabilistic Description

A sequence can be modelled as a **stochastic process**  $\{X_t\}_{t \in \mathcal{T}}$ , where each  $X_t$  is a random variable taking values in  $\mathcal{X}$ , and  $\mathcal{T}$  is the index set (usually  $\mathbb{N}$  or  $\{1, \dots, T\}$ ). The joint distribution over the entire sequence is  $P(X_1 = x_1, \dots, X_T = x_T)$ .

**Definition 0.2** (Stationarity). A stochastic process is **strictly stationary** if for any  $t_1, \dots, t_k$  and any shift  $\tau$ ,

$$P(X_{t_1} \leq x_1, \dots, X_{t_k} \leq x_k) = P(X_{t_1+\tau} \leq x_1, \dots, X_{t_k+\tau} \leq x_k).$$

It is **weakly stationary** (or **covariance stationary**) if:

1.  $\mathbb{E}[X_t] = \mu$  (constant mean),
2.  $\text{Var}(X_t) = \sigma^2$  (constant variance),

3.  $\text{Cov}(X_t, X_{t+\tau})$  depends only on  $\tau$  (the lag).

Stationarity simplifies modeling because the statistical properties do not change over time. Many real-world sequences are non-stationary (e.g., financial returns have time-varying volatility).

### Auto-correlation and Memory

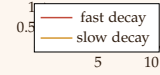
A key characteristic of sequences is that observations are not independent; they exhibit **temporal dependence**.

**Definition 0.3** (Auto-correlation Function (ACF)). For a weakly stationary sequence with variance  $\sigma^2$ , the auto-correlation at lag  $\tau$  is

$$\rho(\tau) = \frac{\text{Cov}(X_t, X_{t+\tau})}{\sigma^2}.$$

The ACF satisfies  $\rho(0) = 1$ ,  $|\rho(\tau)| \leq 1$ , and  $\rho(\tau) = \rho(-\tau)$ .

If  $\rho(\tau)$  decays slowly (e.g.,  $\rho(\tau) \sim e^{-\alpha\tau}$  or power-law), the sequence has **long-term memory**. Short-memory processes have  $\rho(\tau) \rightarrow 0$  exponentially fast.



### Markov Property and Order

A fundamental simplification for sequence modeling is the Markov assumption: the future depends only on a finite past.

**Definition 0.4** (Markov Chain). A discrete-time stochastic process  $\{X_t\}$  is a **Markov chain of order  $k$**  if for all  $t$ ,

$$P(X_t = x_t \mid X_{1:t-1} = x_{1:t-1}) = P(X_t = x_t \mid X_{t-k:t-1} = x_{t-k:t-1}),$$

i.e., the conditional distribution of the next state depends only on the  $k$  most recent states. For  $k = 1$ , it is a **first-order Markov chain**.

**Theorem 0.1** (Markov Chain Decomposition). For a first-order Markov chain, the joint distribution factorizes as

$$P(X_{1:T} = x_{1:T}) = P(X_1 = x_1) \prod_{t=2}^T P(X_t = x_t \mid X_{t-1} = x_{t-1}).$$

The Markov property is the theoretical backbone of many sequence models, including RNNs (which approximate an infinite-order Markov chain via hidden states).

### Sequence Learning Tasks

We categorise learning problems based on inputs and outputs:

1. **Sequence prediction (auto-regressive):** Given past observations  $x_{1:t-1}$ , predict  $x_t$ . The model learns  $P(x_t | x_{1:t-1})$ .
2. **Sequence classification:** Given the entire input sequence  $x_{1:T}$ , predict a single label  $y$ . Example: sentiment analysis.
3. **Sequence labelling (per-time-step classification):** Given  $x_{1:T}$ , predict a label  $y_t$  for each  $t$ . Example: part-of-speech tagging.
4. **Sequence generation:** Sample a new sequence from the learned distribution  $P(x_{1:T})$ .
5. **Sequence-to-sequence (seq2seq):** Map an input sequence  $x_{1:T}$  to an output sequence  $y_{1:T'}$  (possibly of different length). Example: machine translation.

### *Why Feedforward Networks Fail for Sequences*

Feedforward (multilayer perceptron) networks assume fixed-length inputs and outputs, and – critically – they treat each input as independent. To process a sequence of length  $T$ , one could:

1. **Concatenate all elements** into a single  $d \times T$  vector. This fails because  $T$  is variable and the resulting dimension becomes enormous.
2. **Slide a window** of fixed size  $w$  over the sequence, using the last  $w$  values as input to a feedforward network. This ignores dependencies longer than  $w$  (finite memory) and does not share parameters across time positions efficiently.

Moreover, feedforward networks have no built-in mechanism to handle the sequential order or to maintain a state that evolves over time. They cannot capture the inductive bias that the same transformation applies at each time step (weight sharing) – a property that recurrent networks exploit.

### *Recurrence as a Natural Solution*

An RNN addresses these limitations by introducing a **hidden state**  $\mathbf{h}_t$  that evolves over time according to

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \theta),$$

where  $f$  is a parameterised function (same at each step). This gives three crucial properties:

- **Variable-length handling:** the same recurrence applies for any  $T$ .

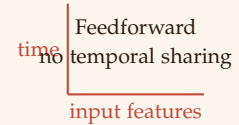


Figure 3: Feedforward networks treat each time step independently; they cannot share weights across time or maintain a dynamic state.

- **Parameter sharing:** the same weights  $\theta$  are used at each time step, reducing the number of parameters and promoting generalization.
- **Memory:** the hidden state  $\mathbf{h}_t$  can theoretically retain information from arbitrarily distant past (though in practice limited by vanishing gradients).

Thus, the RNN is a natural and principled extension of feedforward networks to sequential data. The remainder of this chapter formalises the RNN architecture, training, and its advanced variants.

### Further Reading

For a deeper treatment of stochastic processes and sequence modeling, see:

- Box, G. E. P., Jenkins, G. M., & Reinsel, G. C. (2015). *Time Series Analysis: Forecasting and Control*.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning* (Chapter 13: Sequential Data).
- Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective* (Chapter 17: Markov and Hidden Markov Models).

### Markov Models: From Chains to Hidden States

The Markov property provides a powerful simplification for sequence modeling: the future depends only on a finite past. This idea gives rise to a family of models that are both mathematically tractable and widely used.

#### Discrete-Time Markov Chains

**Definition 0.5** (Markov Chain – First Order). *A discrete-time stochastic process  $\{X_t\}_{t=1}^{\infty}$  with state space  $\mathcal{S}$  (finite or countable) is a **Markov chain** if for all  $t \geq 2$  and all  $s_1, \dots, s_t \in \mathcal{S}$ ,*

$$P(X_t = s_t \mid X_{t-1} = s_{t-1}, \dots, X_1 = s_1) = P(X_t = s_t \mid X_{t-1} = s_{t-1}).$$

The chain is **time-homogeneous** if the transition probability does not depend on  $t$ :

$$P(X_t = j \mid X_{t-1} = i) = T_{ij}, \quad \forall t,$$

where  $\mathbf{T}$  is the **transition matrix** with rows summing to 1.

**Definition 0.6** (Order  $k$  Markov Chain). *A process satisfies the **Markov property of order  $k$**  if*

$$P(X_t \mid X_{1:t-1}) = P(X_t \mid X_{t-k:t-1}).$$

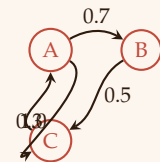


Figure 4: A three-state Markov chain with transition probabilities.

The state can be augmented to  $\tilde{X}_t = (X_{t-k+1}, \dots, X_t)$ , which forms a first-order chain on a larger state space of size  $|\mathcal{S}|^k$ .

#### Key Properties of Markov Chains

- **Stationary distribution:** If a distribution  $\pi$  satisfies  $\pi^\top = \pi^\top \mathbf{T}$ , it is stationary; for an ergodic chain,  $\lim_{n \rightarrow \infty} \mathbf{T}^n$  has rows equal to  $\pi$ .
- **Ergodicity:** A chain that is aperiodic and positive recurrent converges to a unique stationary distribution regardless of initial state.
- **Likelihood of a sequence:**  $P(X_{1:T} = s_{1:T}) = P(X_1 = s_1) \prod_{t=2}^T T_{s_{t-1}, s_t}$ .

#### Hidden Markov Models (HMMs)

In many real-world sequences, the underlying state is not directly observable. HMMs extend Markov chains by assuming that each state  $X_t$  emits an observation  $O_t$  through a known emission distribution.

**Definition 0.7** (Hidden Markov Model). *An HMM is a doubly stochastic process  $(X_t, O_t)_{t=1}^T$  where:*

1.  $\{X_t\}$  is a time-homogeneous Markov chain with state space  $\mathcal{S}$ , initial distribution  $\pi$ , and transition matrix  $\mathbf{T}$ .
2. Given  $X_t$ , the observation  $O_t$  is independent of all other states and observations, with emission probability  $E(o | X_t = s) = P(O_t = o | X_t = s)$ .

The joint distribution of a sequence of states and observations is

$$P(X_{1:T} = s_{1:T}, O_{1:T} = o_{1:T}) = \pi_{s_1} \prod_{t=2}^T T_{s_{t-1}, s_t} \prod_{t=1}^T E(o_t | s_t).$$

#### Inference and Learning in HMMs

Three fundamental problems for HMMs are solved with efficient dynamic programming algorithms.

1. **Likelihood Computation (Forward Algorithm)** Given observations  $o_{1:T}$ , compute  $P(O_{1:T} = o_{1:T}) = \sum_{s_{1:T}} P(X_{1:T}, O_{1:T})$ . The forward recursion:

$$\alpha_1(s) = \pi_s E(o_1 | s), \quad \alpha_t(s) = E(o_t | s) \sum_{s'} \alpha_{t-1}(s') T_{s', s}.$$

Then  $P(O_{1:T}) = \sum_s \alpha_T(s)$ .

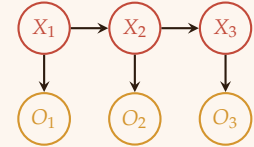


Figure 5: Graphical model of an HMM: hidden states  $X_t$  (circle) emit observations  $O_t$  (square).

2. *Most Likely State Sequence (Viterbi)* Find  $\arg \max_{s_{1:T}} P(X_{1:T} \mid O_{1:T})$ .

The Viterbi algorithm uses:

$$\delta_1(s) = \pi_s E(o_1 \mid s), \quad \delta_t(s) = E(o_t \mid s) \max_{s'} \delta_{t-1}(s') T_{s's},$$

with backpointers to reconstruct the optimal path.

3. *Parameter Estimation (Baum–Welch / EM)* Given observations but no states, we maximise  $\log P(O_{1:T} \mid \theta)$  via the EM algorithm. The E-step computes posterior probabilities  $\gamma_t(s) = P(X_t = s \mid O_{1:T})$  and  $\xi_t(s', s) = P(X_{t-1} = s', X_t = s \mid O_{1:T})$  using the forward–backward algorithm. The M-step updates:

$$\pi_s \propto \gamma_1(s), \quad T_{s's} \propto \frac{\sum_{t=2}^T \xi_t(s', s)}{\sum_{t=2}^T \gamma_{t-1}(s')}, \quad E(o \mid s) \propto \frac{\sum_{t: O_t=o} \gamma_t(s)}{\sum_t \gamma_t(s)}.$$

**Theorem 0.2** (Baum–Welch Convergence). *The Baum–Welch algorithm monotonically increases the likelihood  $P(O_{1:T} \mid \theta)$  and converges to a local maximum of the likelihood.*

### Markov Decision Processes (MDPs) – A Brief Mention

When actions influence state transitions, we obtain an MDP. While not purely generative, MDPs are foundational for reinforcement learning. An MDP is a tuple  $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$  where:

- $\mathcal{S}$ : state space,
- $\mathcal{A}$ : action space,
- $\mathcal{T}(s' \mid s, a)$ : transition probability,
- $\mathcal{R}(s, a)$ : immediate reward,
- $\gamma \in [0, 1]$ : discount factor.

The goal is to find a policy  $\pi(a \mid s)$  that maximises expected cumulative reward.

### From Hidden Markov Models to Recurrent Neural Networks

HMMs and RNNs both address sequence modeling but differ fundamentally:

- **State representation:** HMM states are discrete (finite  $\mathcal{S}$ ); RNN states are continuous vectors  $\mathbf{h}_t \in \mathbb{R}^{d_h}$ . Continuous states allow exponentially more expressive power for a fixed dimensionality.



- **Transition function:** HMM transitions are linear (matrix multiplication on a simplex); RNN transitions are nonlinear (e.g.,  $\tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t)$ ), enabling the modelling of complex dynamics.
- **Emission model:** HMM emissions factorize as  $E(o_t | s_t)$ ; RNNs can produce outputs that depend on the entire history via  $\mathbf{h}_t$ .
- **Learning:** HMMs are trained with EM (Baum–Welch), which guarantees local likelihood increase; RNNs use BPTT and stochastic gradient descent, which scale better to large state spaces and long sequences.
- **Long-range dependencies:** HMMs suffer from exponentially many states to represent  $k$ -th order dependencies; RNNs attempt to compress history into a fixed-size vector (though suffer from vanishing gradients).

**Theorem 0.3** (HMM as a Linear RNN). *If the state vector  $\mathbf{h}_t$  is taken as the vector of probabilities  $P(X_t = s | \text{history})$ , then the update  $\mathbf{h}_t \propto \mathbf{E}_{o_t} \odot (\mathbf{T}^\top \mathbf{h}_{t-1})$  (normalised) is a linear recurrence (with a non-linear normalisation). This is equivalent to the forward algorithm of an HMM.*

The inadequacies of HMMs for large-scale sequence modeling – discrete state, linear transitions, independence of observations given state – motivated the development of more powerful models, including conditional random fields (CRFs) and ultimately recurrent neural networks. However, HMMs remain important for their interpretability, small-sample efficiency, and theoretical guarantees.

### Sequence Modeling and the Need for Recurrence

Many machine learning tasks involve sequential data: time series, natural language text, audio signals, or video frames. In such data, each element depends not only on the current input but also on the history of previous elements. Standard feedforward networks (multilayer perceptrons) assume that inputs are independent, which fails for sequences.

A natural approach is to allow the network to maintain an internal **state** that evolves as it processes the sequence. This state serves as a memory that captures relevant information from the past. Recurrent Neural Networks (RNNs) are the canonical architecture for this purpose.

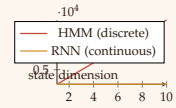


Figure 6: Continuous RNN states can represent exponentially more distinct histories than discrete HMM states of the same dimension.

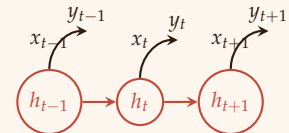


Figure 7: Unrolled RNN: the same recurrent cell is applied at each time step, sharing weights across time.

### Notation for Sequences

Let a sequence be denoted by  $\mathbf{x}_{1:T} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T)$ , where each  $\mathbf{x}_t \in \mathbb{R}^{d_x}$  is the input at time step  $t$ . The corresponding targets (if supervised) are  $\mathbf{y}_{1:T} = (\mathbf{y}_1, \dots, \mathbf{y}_T)$  with  $\mathbf{y}_t \in \mathbb{R}^{d_y}$  (or categorical). In many applications, the target at time  $t$  depends on the entire history up to  $t$ .

### The Elman RNN Architecture

The most basic RNN, proposed by Elman (1990), consists of an input layer, a hidden layer with recurrent connections, and an output layer. At each time step  $t$ , the hidden state  $\mathbf{h}_t \in \mathbb{R}^{d_h}$  is updated based on the current input  $\mathbf{x}_t$  and the previous hidden state  $\mathbf{h}_{t-1}$ .

**Definition 0.8 (RNN Cell).** A simple RNN cell is defined by the following equations:

$$\mathbf{a}_t = \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h, \quad (1)$$

$$\mathbf{h}_t = \phi_h(\mathbf{a}_t), \quad (2)$$

$$\mathbf{o}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y, \quad (3)$$

$$\hat{\mathbf{y}}_t = \phi_y(\mathbf{o}_t), \quad (4)$$

where:

- $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times d_x}$  is the input-to-hidden weight matrix,
- $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$  is the recurrent (hidden-to-hidden) weight matrix,
- $\mathbf{W}_{hy} \in \mathbb{R}^{d_y \times d_h}$  is the hidden-to-output weight matrix,
- $\mathbf{b}_h \in \mathbb{R}^{d_h}$ ,  $\mathbf{b}_y \in \mathbb{R}^{d_y}$  are bias vectors,
- $\phi_h$  is a nonlinear activation function (typically tanh or ReLU),
- $\phi_y$  is the output activation (e.g., softmax for classification, identity for regression).

The initial hidden state  $\mathbf{h}_0$  is usually set to the zero vector, but can also be learned.

### Unrolling Through Time

To understand training, it is convenient to “unroll” the recurrence over time, obtaining a deep feedforward network with shared weights across layers (each layer corresponds to a time step). The unrolled architecture for  $T$  time steps is shown in Figure ??.

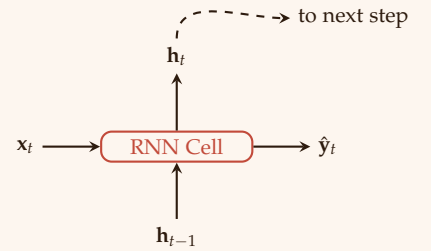


Figure 8: Schematic of a single RNN cell. The hidden state  $\mathbf{h}_t$  is fed back to the next time step.

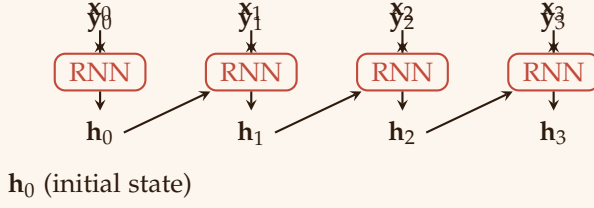


Figure 9: Unrolled RNN for  $T = 4$  time steps. The same weights  $\mathbf{W}_{xh}$ ,  $\mathbf{W}_{hh}$ ,  $\mathbf{W}_{hy}$  are reused at every step.

### Forward Propagation

Given an input sequence  $\mathbf{x}_{1:T}$ , forward propagation computes the hidden states and outputs sequentially:

---

```

1:  $\mathbf{h}_0 \leftarrow \mathbf{0}$  (or learned initial state)
2: for  $t = 1$  to  $T$  do
3:    $\mathbf{a}_t \leftarrow \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h$ 
4:    $\mathbf{h}_t \leftarrow \phi_h(\mathbf{a}_t)$ 
5:    $\mathbf{o}_t \leftarrow \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$ 
6:    $\hat{\mathbf{y}}_t \leftarrow \phi_y(\mathbf{o}_t)$ 
7: end for
    
```

---

Algorithm 1: Forward pass of an RNN

### Training RNNs: Backpropagation Through Time (BPTT)

RNNs are trained by minimising a loss function that aggregates errors over all time steps. For a sequence with targets  $\mathbf{y}_{1:T}$ , the total loss is the sum of per-time-step losses:

$$\mathcal{L}(\mathbf{y}_{1:T}, \hat{\mathbf{y}}_{1:T}) = \sum_{t=1}^T \ell(\mathbf{y}_t, \hat{\mathbf{y}}_t),$$

where  $\ell$  is a pointwise loss (e.g., cross-entropy or squared error). Because the same parameters are shared across time, we must back-propagate gradients through the entire unrolled network. This procedure is called **backpropagation through time (BPTT)**.

### Gradient Computation

Let  $\theta$  denote any trainable parameter (e.g.,  $w_{ij}^{(xh)}$ ). The total derivative is

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{t=1}^T \frac{\partial \ell_t}{\partial \theta},$$

where  $\ell_t$  is the loss at time  $t$ . For parameters that appear at every time step (like  $\mathbf{W}_{hh}$ ), the contribution from each  $t$  depends on all future time steps because the hidden state at time  $t$  influences all subsequent losses.

We derive the gradients for  $\mathbf{W}_{hh}$ ; similar reasoning applies to  $\mathbf{W}_{xh}$ ,  $\mathbf{b}_h$ ,  $\mathbf{W}_{hy}$ ,  $\mathbf{b}_y$ .

*Error Signals* Define the error at the output of the RNN at time  $t$  as:

$$\delta_t^{\text{out}} = \frac{\partial \ell_t}{\partial \mathbf{o}_t} = \nabla_{\mathbf{y}_t} \ell_t \odot \phi'_y(\mathbf{o}_t).$$

For the hidden layer, we need the sensitivity of the total future loss with respect to the pre-activation  $\mathbf{a}_t$ . By the chain rule,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_t} = \underbrace{\frac{\partial \ell_t}{\partial \mathbf{a}_t}}_{\text{immediate}} + \underbrace{\frac{\partial \mathcal{L}_{t+1:T}}{\partial \mathbf{a}_t}}_{\text{from future}}.$$

The immediate contribution is  $\frac{\partial \ell_t}{\partial \mathbf{a}_t} = \mathbf{W}_{hy}^T \delta_t^{\text{out}} \odot \phi'_h(\mathbf{a}_t)$ .

The future contribution comes from the fact that  $\mathbf{h}_t$  affects  $\mathbf{a}_{t+1}$  via  $\mathbf{W}_{hh}\mathbf{h}_t$ . Hence,

$$\frac{\partial \mathcal{L}_{t+1:T}}{\partial \mathbf{a}_t} = \left( \frac{\partial \mathbf{a}_{t+1}}{\partial \mathbf{h}_t} \right)^T \frac{\partial \mathcal{L}_{t+1:T}}{\partial \mathbf{a}_{t+1}} = \mathbf{W}_{hh}^T \delta_{t+1} \odot \phi'_h(\mathbf{a}_t),$$

where we define  $\delta_t = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_t}$ . Thus, the backward recurrence is:

$$\delta_T = \mathbf{W}_{hy}^T \delta_T^{\text{out}} \odot \phi'_h(\mathbf{a}_T), \quad (5)$$

$$\delta_t = \left( \mathbf{W}_{hy}^T \delta_t^{\text{out}} + \mathbf{W}_{hh}^T \delta_{t+1} \right) \odot \phi'_h(\mathbf{a}_t), \quad t = T-1, \dots, 1. \quad (6)$$

*Gradients with Respect to Weights* Once the  $\delta_t$  are known, the gradients are:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \delta_t \mathbf{h}_{t-1}^T, \quad (7)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{xh}} = \sum_{t=1}^T \delta_t \mathbf{x}_t^T, \quad (8)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_h} = \sum_{t=1}^T \delta_t, \quad (9)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^T \delta_t^{\text{out}} \mathbf{h}_t^T, \quad (10)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_y} = \sum_{t=1}^T \delta_t^{\text{out}}. \quad (11)$$

These gradients are then used in any gradient-based optimiser (SGD, Adam, etc.). The complete BPTT algorithm is summarised below.

Algorithm 2: Backpropagation Through Time (BPTT)

- 
- 1: Run forward pass to store  $\mathbf{a}_t, \mathbf{h}_t, \mathbf{o}_t$  for  $t = 1, \dots, T$ .
  - 2: Compute  $\delta_T^{\text{out}} = \nabla_{\hat{\mathbf{y}}_T} \ell_T \odot \phi'_y(\mathbf{o}_T)$ .
  - 3:  $\delta_T = \mathbf{W}_{hy}^T \delta_T^{\text{out}} \odot \phi'_h(\mathbf{a}_T)$ .
  - 4: **for**  $t = T - 1$  down to 1 **do**
  - 5:      $\delta_t^{\text{out}} = \nabla_{\hat{\mathbf{y}}_t} \ell_t \odot \phi'_y(\mathbf{o}_t)$
  - 6:      $\delta_t = (\mathbf{W}_{hy}^T \delta_t^{\text{out}} + \mathbf{W}_{hh}^T \delta_{t+1}) \odot \phi'_h(\mathbf{a}_t)$
  - 7: **end for**
  - 8: Accumulate gradients for  $\mathbf{W}_{hh}, \mathbf{W}_{xh}, \mathbf{b}_h, \mathbf{W}_{hy}, \mathbf{b}_y$  using the formulas above.
  - 9: Update parameters with gradient descent.
- 

### Vanishing and Exploding Gradients

A fundamental difficulty in training RNNs on long sequences is the problem of **vanishing** or **exploding gradients**. Observe that the recurrence for  $\delta_t$  involves multiplication by  $\mathbf{W}_{hh}^T$  and  $\phi'_h(\mathbf{a}_t)$ . Unrolling further, the gradient contains terms like  $(\mathbf{W}_{hh}^T)^k \prod_{i=t+1}^{t+k} \phi'_h(\mathbf{a}_i)$ .

If the eigenvalues of  $\mathbf{W}_{hh}$  are less than 1 and the activation derivatives are also  $< 1$  (typical for sigmoid or tanh), then long-range gradients decay exponentially to zero (vanishing). Conversely, if eigenvalues exceed 1, gradients explode. Vanishing gradients prevent the network from learning long-range dependencies; exploding gradients make training numerically unstable.

**Theorem 0.4** (Vanishing Gradients in RNNs). *For a simple RNN with  $\phi_h = \tanh$ , let  $\gamma = \max_i |\lambda_i(\mathbf{W}_{hh})|$  and  $\rho = \max_z |\tanh'(z)| = 1$ . If  $\gamma < 1$ , then the norm of the gradient contribution from time  $t$  to time  $t - k$  decays as  $\mathcal{O}(\gamma^k)$ .*

Solutions include:

- Gradient clipping (for exploding gradients): rescale gradients if their norm exceeds a threshold.
- Careful weight initialisation (e.g., orthogonal initialisation).
- Gated architectures (LSTM, GRU) that allow gradients to flow more easily.

### Gated Recurrent Architectures

To mitigate the vanishing gradient problem, more sophisticated RNN cells introduce **gates** that control the flow of information.

### Long Short-Term Memory (LSTM)

The LSTM, proposed by Hochreiter and Schmidhuber (1997), maintains a separate cell state  $\mathbf{c}_t$  in addition to the hidden state  $\mathbf{h}_t$ . The cell state acts as a “conveyor belt” that can carry information over many steps. Three gates control access:

- **Forget gate:**  $\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$
- **Input gate:**  $\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$
- **Output gate:**  $\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$

A candidate cell update is  $\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$ . Then the cell state and hidden state are updated:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad (12)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \quad (13)$$

The forget gate allows the network to selectively erase the past; the input gate decides which new information to store; the output gate controls what is exposed as the hidden state.

### Gated Recurrent Unit (GRU)

The GRU (Cho et al., 2014) simplifies the LSTM by combining the forget and input gates into a single **update gate** and merging the cell and hidden states. Its equations are:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z) \quad (\text{update gate}), \quad (14)$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r) \quad (\text{reset gate}), \quad (15)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h), \quad (16)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \quad (17)$$

GRUs have fewer parameters than LSTMs and often perform similarly.

### Connection to Maximum Likelihood Estimation

For sequence prediction (e.g., language modeling), the RNN defines a conditional probability distribution over the next token given the history:

$$P(\mathbf{y}_t \mid \mathbf{x}_{1:t}, \mathbf{y}_{1:t-1}) = \text{softmax}(\mathbf{o}_t).$$

The total likelihood of a sequence is  $\prod_{t=1}^T P(\mathbf{y}_t \mid \text{history})$ . Minimising the cross-entropy loss is therefore equivalent to maximum likelihood estimation under the model.

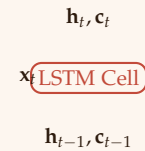


Figure 10: Schematic of an LSTM cell. The cell state  $\mathbf{c}_t$  (not shown separately) flows through the top, modulated by gates.

## Practical Considerations and Extensions

- **Truncated BPTT:** For very long sequences, full BPTT is expensive. Truncated BPTT splits the sequence into blocks and only back-propagates a fixed number of steps ( $k$ ) inside each block. This approximates the true gradient and is widely used.
- **Teacher forcing:** During training, the true previous target  $\mathbf{y}_{t-1}$  (or a sample) is fed as input instead of the model's own prediction. This speeds up training but may cause exposure bias.
- **Deep RNNs:** Stacking multiple RNN layers (each layer's hidden state becomes input to the next layer) increases representational capacity.
- **Bidirectional RNNs:** Two RNNs process the sequence forward and backward, and their hidden states are concatenated. This provides context from both past and future, useful for sequence tagging.

---

```

1: Initialise  $\mathbf{h}_0$ .
2: for each block  $b$  of  $k$  consecutive time steps do
3:   Run forward pass for  $k$  steps, storing intermediate values.
4:   Compute loss for the block.
5:   Backpropagate gradients only through the  $k$  steps (do not
      propagate beyond the block start).
6:   Update parameters.
7:   Carry over  $\mathbf{h}_{b,k}$  as initial state for the next block.
8: end for
    
```

---

Algorithm 3: Truncated BPTT (with block size  $k$ )

## Summary

Recurrent Neural Networks provide a principled framework for modeling sequential data by maintaining an internal state that evolves over time. The training procedure, backpropagation through time, is a direct extension of the backpropagation algorithm for feedforward networks but must handle weight sharing across time. The vanishing and exploding gradient problems are inherent to simple RNNs; gated architectures (LSTM, GRU) are the standard solution. These models form the basis of modern sequence-to-sequence learning, machine translation, speech recognition, and many other applications.

The mathematical tools developed in earlier chapters – probability, MLE, gradient descent, and information theory – all play a role in understanding and training RNNs. The next logical step is to explore

attention mechanisms and transformers, which have largely replaced RNNs in many domains but still rely on the same foundational concepts.